

Serverless Smart Attendance System with Face Recognition

Technical Design Document

➤ **Region:** ap-south-1

➤ **Prepared By:**

- Swagata Sinha
- D Mohamed Fadhil
- M.T Nishanth

▪ **Document Structure:**

1. Executive Summary
2. Project Overview
3. Business Problem
4. Functional Requirements
5. Non-Functional Requirements
6. High-Level Architecture
7. Component Design
8. Data Model Design
9. Lambda Function URL Specification
10. Face Recognition Logic
11. Attendance Processing Logic
12. Daily Summary Design
13. Weekly Analytics Design
14. Security Design and IAM Permission Matrix
15. CloudWatch Logging Strategy
16. Failure Handling and Recovery

17. Team Split and Integration Agreement
 18. Deployment Runbook
 19. Acceptance Tests
 20. Cost Considerations
 21. Future Enhancements
 22. Conclusion
-

1. Executive Summary

The **Serverless Smart Attendance System with Face Recognition** is a fully serverless AWS-based attendance management solution that automates employee clock-in and clock-out using facial recognition technology.

The system allows an employee to capture a face image through a web browser or mobile device and upload it securely to Amazon S3 using a pre-signed URL. An S3 upload event automatically triggers an AWS Lambda function that invokes Amazon Rekognition to identify the employee by matching the uploaded image against a registered face collection.

Once the employee is identified, attendance is recorded automatically in Amazon DynamoDB according to predefined attendance rules. The first successful recognition creates the employee's clock-in time, while the second successful recognition updates the clock-out time. Invalid images, multiple faces, low-quality images, or unknown faces are rejected automatically.

The solution also includes scheduled EventBridge jobs that generate daily attendance summaries and weekly attendance analytics without manual intervention. Reports are stored in Amazon S3 and can be viewed through an administrator dashboard.

The architecture demonstrates real cloud engineering concepts including event-driven processing, serverless computing, facial recognition, automated scheduling, least-privilege IAM security, DynamoDB integration, lifecycle management, and centralized monitoring through Amazon CloudWatch.

The solution eliminates manual attendance registers, reduces proxy attendance, improves operational efficiency, and provides a scalable, secure, and cost-effective attendance management platform suitable for organizations of varying sizes.

2. Project Overview

The Smart Attendance System with Face Recognition is a serverless AWS-based application that automates employee attendance using facial recognition technology. Employees capture and upload their face images to Amazon S3 through a pre-signed URL, after which AWS Lambda and Amazon Rekognition identify the employee and record attendance automatically in Amazon DynamoDB.

The system supports automatic clock-in and clock-out, daily attendance summary generation, weekly analytics reporting, and an admin dashboard for attendance monitoring. Built entirely on managed AWS services, the solution is scalable, secure, cost-effective, and eliminates the need for manual attendance tracking or dedicated servers.

3. Business Problem

Employee attendance is one of the most critical operational processes in every organization. Traditional attendance methods such as paper registers, RFID cards, biometric fingerprint devices, or manual spreadsheets often suffer from issues including proxy attendance, human errors, hardware dependency, and time-consuming record maintenance.

Manual attendance systems become increasingly difficult to manage as the number of employees grows. They require administrative effort for verification, reporting, and absence calculation, while also increasing the risk of inaccurate attendance records.

Biometric fingerprint systems require physical contact and dedicated hardware, resulting in maintenance costs and hygiene concerns. Card-based systems can be misused through card sharing, leading to inaccurate attendance tracking.

Organizations require a contactless, automated, and scalable attendance solution that minimizes manual intervention while providing accurate and auditable records.

This project addresses these challenges by leveraging AWS serverless services and Amazon Rekognition to automatically identify employees using facial recognition technology. The event-driven architecture enables real-time attendance processing, automated report generation, and centralized monitoring without maintaining dedicated servers.

The solution improves attendance accuracy, eliminates proxy attendance, reduces administrative workload, and provides a secure and scalable cloud-native attendance management platform.

4. Functional Requirements

The Smart Attendance System shall provide the following functional capabilities:

- Register employee face images into an Amazon Rekognition Collection.
- Maintain employee information in Amazon DynamoDB.
- Generate secure pre-signed URLs for direct image upload to Amazon S3.
- Capture employee face images through a web browser or mobile interface.
- Upload captured images directly to Amazon S3.
- Automatically trigger attendance processing using S3 Object Created events.
- Detect and validate face quality before recognition.

- Search registered employee faces using Amazon Rekognition SearchFacesByImage.
 - Reject invalid images containing no face or multiple faces.
 - Reject images with similarity scores below the configured threshold.
 - Automatically create employee clock-in records.
 - Automatically update employee clock-out records on second recognition.
 - Ignore duplicate attendance scans after clock-out has been recorded.
 - Store recognition results for frontend display.
 - Generate daily attendance summaries automatically.
 - Mark employees absent if no attendance record exists by end of day.
 - Generate weekly attendance analytics reports.
 - Allow administrators to query attendance records using filters.
 - Store reports securely in Amazon S3.
 - Capture operational logs through Amazon CloudWatch.
-

5. Non-Functional Requirements

The Smart Attendance System should satisfy the following quality attributes:

- **Scalability-** The architecture shall support increasing numbers of employees without requiring infrastructure changes by leveraging AWS Lambda and managed serverless services.
 - **Reliability-** Attendance processing should execute automatically with minimal downtime using event-driven triggers and managed AWS infrastructure.
 - **Performance-** Attendance recognition results should be displayed to users within approximately three seconds under normal operating conditions.
 - **Security-** All AWS resources shall follow the principle of least privilege. Images should remain private and attendance data should be securely stored using encrypted AWS services.
 - **Availability-** The system should remain operational without dependency on continuously running servers by using fully managed AWS services.
 - **Auditability-** Every successful recognition event and attendance update should be stored in DynamoDB for future verification and reporting.
 - **Maintainability-** Each AWS service should perform a clearly defined responsibility, making the architecture modular and easy to extend.
 - **Cost Efficiency-** The serverless architecture should incur costs only when attendance events or scheduled analytics are executed, minimizing infrastructure expenses.
-

6. High-Level Architecture

The Smart Attendance System follows a fully serverless, event-driven architecture.

- 1) The employee opens the attendance webpage using a browser or mobile device.
- 2) The frontend requests a pre-signed upload URL from the Generate Pre-signed URL Lambda Function.
- 3) The Lambda function generates a secure temporary upload URL for Amazon S3.
- 4) The employee captures a face image and uploads it directly to the Images Bucket in Amazon S3.
- 5) The S3 upload event automatically invokes the Attendance Processor Lambda.
- 6) The Attendance Processor validates image quality and detects faces using Amazon Rekognition.
- 7) Rekognition searches the registered face collection to identify the employee.
- 8) Employee information is retrieved from the Employees DynamoDB table.
- 9) Attendance logic determines whether the recognition represents Clock-In, Clock-Out, Late Arrival, or Invalid Recognition.
- 10) Attendance records are stored in the Attendance DynamoDB table.
- 11) Recognition results are stored in the Recognition Results DynamoDB table for frontend retrieval.
- 12) The frontend displays attendance confirmation to the employee.
- 13) Amazon EventBridge executes the Daily Attendance Processor every night to mark absent employees and generate attendance summaries.
- 14) Amazon EventBridge executes the Weekly Analytics Generator every Sunday to generate attendance statistics.
- 15) Daily summaries and weekly analytics reports are stored inside Amazon S3 Reports Bucket.
- 16) Administrators retrieve attendance records through the Admin Query Lambda for dashboard visualization.

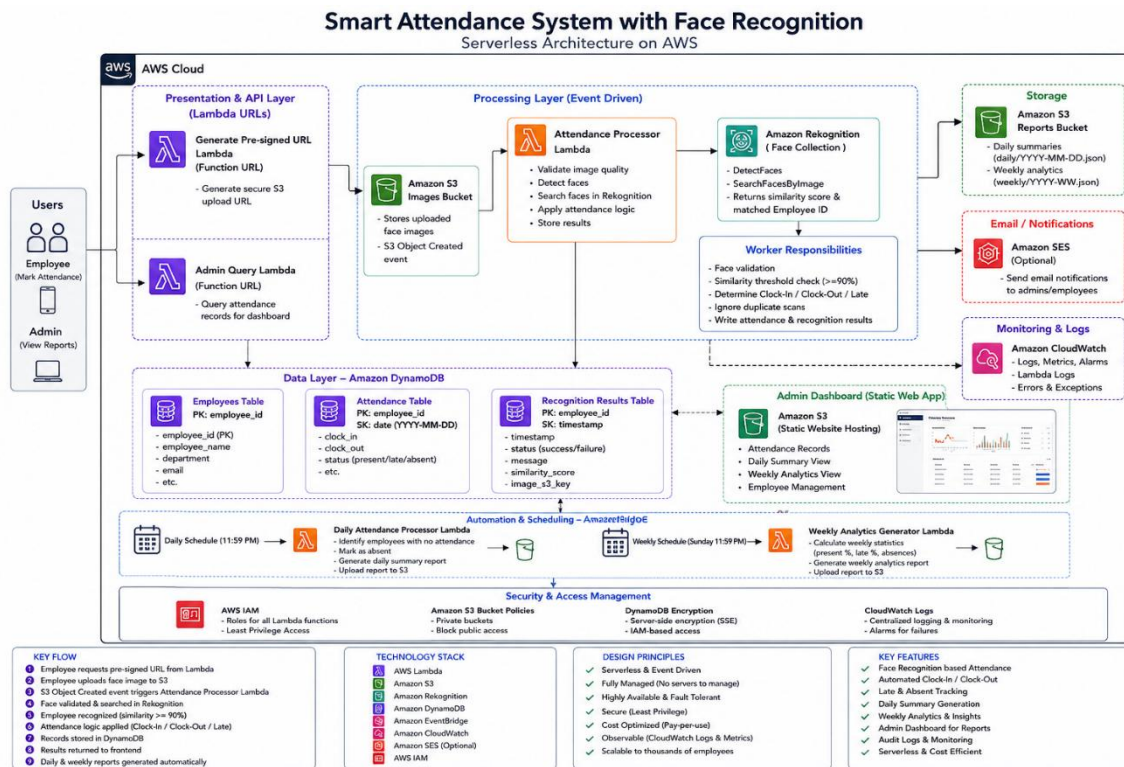


Figure 1: AWS Architecture of Smart Attendance System with Face Recognition

7. Detailed Component Design

7.1 Amazon S3

Responsibilities:

- Store uploaded employee face images.
- Trigger attendance processing automatically.
- Store generated daily attendance reports.
- Store weekly analytics reports.
- Host the static frontend website.

Bucket Structure:

- sas-images-bucket
- sas-reports-bucket
- sas-frontend-bucket

Lifecycle rules automatically delete uploaded images after one hour to reduce storage cost and improve privacy.

7.2 AWS Lambda

The system contains multiple Lambda functions.

Generate Pre-signed URL Lambda

- Generates secure upload URLs.
- Prevents public write access to S3.
- Returns upload URL and object key.

Attendance Processor Lambda

- Triggered by S3 upload events.
- Validates image quality.
- Detects faces.
- Calls Rekognition SearchFacesByImage.
- Applies attendance business rules.
- Stores attendance records.

Admin Query Lambda

- Reads attendance records.
- Applies employee/date/status filters.
- Returns JSON responses for dashboard display.

Daily Attendance Processor Lambda

- Executes daily at 11:59 PM.
- Identifies employees without attendance.
- Marks absences.
- Generates daily summary reports.

Weekly Analytics Generator Lambda

- Executes every Sunday.
- Calculates attendance percentages.
- Computes late percentages.
- Generates weekly analytics reports.

7.3 Amazon Rekognition

Amazon Rekognition provides AI-powered facial recognition.

Responsibilities:

- Detect faces in uploaded images.
- Validate image quality.
- Compare uploaded faces against registered employees.
- Return similarity score.
- Reject unknown faces.

The Rekognition Collection stores employee faces indexed using employee_id as ExternalImageId.

7.4 Amazon DynamoDB

DynamoDB acts as the primary database for the system.

It stores:

- Employee master records
- Attendance records
- Recognition results

The database provides low-latency access and automatic scalability.

7.5 Amazon EventBridge

EventBridge schedules automated background processing.

Responsibilities:

- Trigger Daily Attendance Processor.
 - Trigger Weekly Analytics Generator.
 - Eliminate manual scheduling.
-

7.6 Amazon CloudWatch

CloudWatch provides monitoring and logging.

It records:

- Lambda execution logs
- Attendance failures

- Rekognition failures
 - Processing duration
 - Lambda invocations
 - System exceptions
-

7.7 AWS IAM

IAM secures the architecture through least-privilege access.

Permissions are assigned separately for:

- Lambda execution
- DynamoDB access
- S3 access
- Rekognition access
- CloudWatch logging

No component receives unnecessary permissions.

8. Data Model Design

The system uses three DynamoDB tables to separate employee information, attendance records, and recognition logs.

Employees Table

Table Name: sas-employees-table

Partition Key: employee_id

Example Record

```
{  
  "employee_id": "EMP001",  
  "employee_name": "Riya Sen"  
}
```

Purpose:

- Employee lookup
- Face mapping

- Attendance validation

Attendance Table

Table Name: sas-attendance-table

Partition Key: employee_id

Sort Key: date

Example Record

```
{  
  "employee_id": "EMP001",  
  "date": "2026-06-11",  
  "clock_in": "08:55:00",  
  "clock_out": "17:10:00",  
  "status": "present"  
}
```

Purpose:

- Daily attendance records
- Clock-in tracking
- Clock-out tracking
- Absence calculation

Recognition Results Table

Table Name: sas-recognition-results-table

Partition Key: employee_id

Sort Key: timestamp

Example Record

```
{  
  "employee_id": "EMP001",  
  "employee_name": "Ritu Sen",  
  "timestamp": "2026-06-11T08:55:00",  
}
```

```
"status": "present",  
"success": true  
}
```

Purpose:

- Store recognition history
 - Display frontend attendance results
 - Maintain audit logs for successful recognitions
-

9. Lambda Function URL Specification

The frontend communicates directly with AWS Lambda Function URLs. No API Gateway is used in this project.

The Lambda Function URLs provide secure HTTPS endpoints for generating pre-signed upload URLs and retrieving attendance records from the backend.

Function 1 – Generate Pre-signed Upload URL

Lambda Name:

sas-generate-presigned-url

Invocation:

Lambda Function URL (HTTP GET)

Purpose:

Generate a temporary Amazon S3 pre-signed URL that allows the employee to upload a captured face image directly to the Images Bucket.

Response Example

```
{  
  "uploadUrl": "https://signed-url",  
  "key": "uploads/image001.jpg"  
}
```

The frontend uploads the captured image directly to Amazon S3 using the returned upload URL.

Function 2 – Admin Attendance Query

Lambda Name:

sas-admin-query

Invocation:

Lambda Function URL (HTTP GET)

Purpose:

Retrieve attendance records for the administrator dashboard.

Supported Filters:

- employee_id
- date range
- attendance status

Response Example

```
[  
  {  
    "employee_id": "EMP001",  
    "employee_name": "Ritu Sen",  
    "date": "2026-06-11",  
    "clock_in": "08:55:00",  
    "clock_out": "17:10:00",  
    "status": "present"  
  }  
]
```

The frontend dashboard uses this Lambda Function URL to display attendance records without requiring API Gateway.

Event-Driven Attendance Processing

Unlike traditional REST APIs, attendance processing is fully event-driven.

1. Employee requests an upload URL from the **Generate Pre-signed URL Lambda**.
2. The employee uploads the captured image directly to **Amazon S3**.
3. The **S3 Object Created Event** automatically triggers the **Attendance Processor Lambda**.
4. The Lambda invokes **Amazon Rekognition SearchFacesByImage**.
5. Employee information is retrieved from **DynamoDB**.
6. Attendance records are created or updated automatically.

7. Recognition results are stored for frontend display.

This architecture removes the need for API Gateway and keeps the entire attendance workflow fully serverless and event-driven.

10. Face Recognition Algorithm Flow

The Attendance Processor Lambda performs the following sequence after an image upload.

- 1) Receive S3 Object Created Event.
- 2) Download uploaded image.
- 3) Detect faces using Amazon Rekognition.
- 4) Validate that exactly one face exists.
- 5) Perform image quality validation.
- 6) Reject blurry or poor-quality images.
- 7) Invoke SearchFacesByImage API.
- 8) Compare uploaded face against Rekognition Collection.
- 9) Verify similarity score is greater than or equal to 90%.
- 10) Retrieve employee_id from ExternalImageId.
- 11) Lookup employee information from DynamoDB.
- 12) Return employee identity.
- 13) Continue attendance processing.

If any validation fails, attendance is rejected and the recognition result is stored as unsuccessful.

11. Attendance Processing Logic

Attendance follows deterministic business rules.

Rule 1

If no attendance record exists for the current date:

- Create attendance record.
- Store clock_in.
- Determine attendance status.

Status calculation:

- $\text{clock_in} \leq 09:00 \text{ AM} \rightarrow \text{Present}$
 - $\text{clock_in} > 09:00 \text{ AM} \rightarrow \text{Late}$
-

Rule 2

If attendance record exists but clock_out is empty:

- Update clock_out.
 - Keep previous attendance status.
-

Rule 3

If both clock_in and clock_out already exist:

Ignore additional scans.

Rule 4

If employee cannot be recognized:

Return

```
{  
  "success": false,  
  "message": "Employee Not Recognized"  
}
```

No attendance record is created.

12. Daily Summary Design

Amazon EventBridge automatically invokes the Daily Attendance Processor Lambda every day at 11:59 PM.

The Lambda performs the following operations:

- Read Employees table.
- Read Attendance table.
- Identify employees with no attendance record.
- Mark missing employees as absent.

- Generate daily attendance report.
- Upload report to Amazon S3.

Storage Location:

daily/YYYY-MM-DD.json

Example:

```
[  
  {  
    "employee_id": "EMP001",  
    "employee_name": "Ritu Sen",  
    "clock_in": "08:55:00",  
    "clock_out": "17:10:00",  
    "status": "present"  
  }  
]
```

13. Weekly Analytics Design

Amazon EventBridge invokes the Weekly Analytics Generator every Sunday at 11:59 PM.

The Lambda calculates:

- Present Percentage
- Late Percentage
- Total Absences
- Weekly Attendance Statistics

Reports are uploaded to:

weekly/YYYY-WW.json

Example:

```
[  
  {  
    "employee_id": "EMP001",  
    "employee_name": "Ritu Sen",
```

```
"present_percentage":80,  
"late_percentage":20,  
"absences":1  
}  
]
```

14. Security Design and IAM Permission Matrix

The solution follows AWS security best practices and the principle of least privilege.

Security Features

- Private Amazon S3 buckets
- S3 Versioning enabled
- Lifecycle deletion of images after one hour
- Encrypted DynamoDB tables
- Rekognition access restricted to Lambda
- IAM least privilege
- CloudWatch audit logging

IAM Permission Matrix:

Component	Read Permissions	Write Permissions	Purpose
Generate URL Lambda	None	S3 Presigned URL	Upload URL generation
Attendance Lambda	S3, Rekognition, DynamoDB	Attendance Table, Recognition Table	Attendance processing
Admin Query Lambda	DynamoDB	None	Dashboard queries
Daily Processor	Employees Table, Attendance Table	Attendance Table, Reports Bucket	Daily summary
Weekly Analytics	Attendance Table	Reports Bucket	Weekly analytics
Frontend	None	None	Uses APIs only

15. CloudWatch Logging Strategy

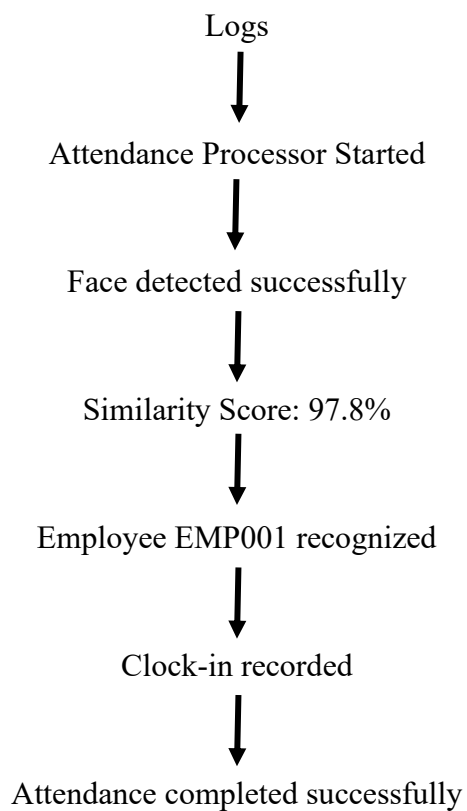
CloudWatch serves as the centralized monitoring platform.

The following events should be logged:

- Image uploaded
- Lambda invocation started

- Face detection success
- Face validation failure
- Rekognition similarity score
- Employee recognized
- Attendance created
- Clock-out updated
- Daily report generated
- Weekly analytics generated
- Processing failures
- Exception stack traces

Example:



16. Failure Handling and Recovery

The attendance system distinguishes between multiple failure scenarios.

Failure Scenario	Expected Behavior
No face detected	Reject attendance
Multiple faces detected	Reject attendance
Low quality image	Reject attendance
Similarity below threshold	Reject attendance
Employee not found	Reject attendance
Lambda execution failure	Retry automatically
DynamoDB temporary failure	Retry request
EventBridge failure	Next scheduled execution

All failures are logged in CloudWatch for troubleshooting.

17. Team Split and Integration Agreement

Team 1

Frontend & User Interface

Responsibilities:

- Camera access
 - Image capture
 - Upload progress
 - Attendance result
 - Admin dashboard
-

Team 2

Recognition & Attendance Backend

Responsibilities:

- Face registration
 - Generate Pre-signed URL Lambda
 - Attendance Processor Lambda
 - Admin Query Lambda
 - Rekognition integration
 - Documentation
-

Team 3

Analytics & Operations

Responsibilities:

- Daily Attendance Processor
- Weekly Analytics Generator
- CloudWatch monitoring
- Lifecycle policies
- Cost analysis

All teams must follow the same DynamoDB schema, bucket names, Lambda names, attendance rules, JSON contracts, and deployment sequence.

18. Deployment Runbook

1. Create Amazon S3 buckets.
2. Create DynamoDB tables.
3. Create Amazon Rekognition Collection.
4. Create IAM Roles.
5. Deploy Lambda Functions.
6. Configure Lambda Function URLs.
7. Configure S3 Event Triggers.
8. Configure EventBridge schedules.
9. Register employee faces.
10. Deploy frontend website.
11. Execute integration testing.
12. Validate CloudWatch logs.
13. Verify daily and weekly report generation.

19. Acceptance Tests

Test Case	Expected Result
Employee before 09:00	Present
Employee after 09:00	Late
Second scan	Clock-out updated

Third scan	Ignored
Unknown face	Employee Not Recognized
No attendance all day	Absent
Daily EventBridge execution	Daily report generated
Weekly EventBridge execution	Weekly analytics generated

20. Cost Considerations

The architecture is highly cost-efficient because all components are serverless.

AWS Service	Cost Characteristic
Amazon S3	Storage + Requests
AWS Lambda	Pay per invocation
DynamoDB	On-demand pricing
Amazon Rekognition	Pay per recognition request
EventBridge	Low-cost scheduled events
CloudWatch	Log storage and monitoring
IAM	No additional cost

For an organization with approximately **100 employees performing two attendance scans per day**, the monthly operational cost remains low due to AWS pay-as-you-go pricing and automatic scaling.

Appendix A – DynamoDB Access Patterns

Access Pattern	Table
Employee Lookup	Employees
Attendance Lookup	Attendance
Recognition History	Recognition Results
Daily Report Generation	Attendance
Weekly Analytics	Attendance

Appendix B – Resource Names

- Rekognition Collection: **sas-employee-face-collection**
- Employees Table: **sas-employees-table**
- Attendance Table: **sas-attendance-table**
- Recognition Results Table: **sas-recognition-results-table**
- Images Bucket: **sas-images-bucket**

- Reports Bucket: **sas-reports-bucket**
- Frontend Bucket: **sas-frontend-bucket**

Appendix C – Glossary

Term	Meaning
Clock-In	First successful attendance scan
Clock-Out	Second successful attendance scan
Face Collection	Rekognition database of registered faces
ExternalImageId	Employee ID stored inside Rekognition
Pre-signed URL	Temporary secure upload URL
EventBridge	AWS scheduling service
Lifecycle Policy	Automatic deletion rule
Serverless	No server management required
Similarity Score	Confidence percentage returned by Rekognition
Attendance Status	Present, Late, or Absent

21. Future Enhancements

- Develop a mobile application for attendance.
- Integrate the system with payroll and HRMS.
- Add real-time attendance analytics dashboards.
- Improve face recognition accuracy and security.
- Enable employee self-service attendance history.

22. Conclusion

The Serverless Smart Attendance System with Face Recognition demonstrates a complete cloud-native attendance management solution built entirely on AWS managed services. The architecture combines Amazon S3, AWS Lambda, Amazon Rekognition, DynamoDB, EventBridge, IAM, and CloudWatch into a scalable, secure, and event-driven platform capable of automating employee attendance with minimal operational overhead.

The solution provides accurate face recognition, automatic attendance recording, scheduled reporting, centralized monitoring, and cost-efficient serverless execution, making it an ideal demonstration project for cloud engineering and AWS solutions architecture.